

Amazon PostgreSQL Blue/Green Deployment for Version Upgrade

To Begin with the Lab

Summary of the Lab

In this lab, an **Amazon Aurora PostgreSQL** environment was upgraded using **Blue/Green Deployment** to keep downtime low and the process predictable. A blue environment was prepared first, and a small client script was used to measure uptime and observe the switchover window. AWS then created a green environment with the newer database version and synchronized the data from the blue environment. The lab showed that the blue environment remained the production endpoint until the switchover was triggered manually. Finally, the upgrade was completed and the application connected to the new version with only a short downtime window.

Understanding Amazon Aurora Blue/Green Deployment

Amazon Aurora Blue/Green Deployment is a safe upgrade method that creates a second, synchronized environment before switching production traffic. The blue environment is the current production database, and the green environment is the upgraded copy that you test first. This approach exists so you can validate the new version before changing production, instead of upgrading in place and risking a longer outage. It is not zero downtime, but it usually gives only a few seconds to a minute of interruption, which is much easier to manage.

Example:

A PostgreSQL database is running in production with a table named employees and another table named employees_new. The green environment is created with the upgraded version, the data is kept in sync, and then the switchover is performed after testing. After the switchover, the same tables are still available, but they are now served by the new version.

Lab Steps

Step 1 – Prepare the Blue Production Database

- Sign in to the AWS Management Console.
- Open **Amazon RDS**.
- I will create the Free Tier Database for lab purpose.
- Select **Aurora PostgreSQL** or **Amazon RDS PostgreSQL** based on the lab setup.

aws [Search] [Alt+S] United States (N. Virginia) cloudriteshk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Availability zone group [info](#)
 Availability zone group
 Select the availability zone where you want to create your database.
 US East (N. Virginia)

Engine options
 Engine type [info](#)

☐ Aurora (MySQL Compatible) ☐ Aurora (PostgreSQL Compatible) ☐ MySQL ☒ PostgreSQL
☐ MariaDB ☐ Oracle ☐ Microsoft SQL Server ☐ IBM Db2

Choose a database creation method

☒ Full configuration
 You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ Easy create
 Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

- Choose the current production version.

aws [Search] [Alt+S] United States (N. Virginia) cloudriteshk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Templates
 Choose a sample template to meet your use case.

☐ Production
 Use defaults for high availability and fast, consistent performance.

☐ Dev/Test
 This instance is intended for development use outside of a production environment.

☒ Free tier
 Use the RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. [Info](#)

Availability and durability
 Deployment options [info](#)
 Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. [Learn more about the Amazon RDS service-level agreement \(SLA\).](#)

☐ Multi-AZ DB cluster deployment (3 instances)
 Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:

- 99.99% uptime
- Redundancy across Availability Zones
- Increased read capacity
- Reduced write latency

☐ Multi-AZ DB instance deployment (2 instances)
 Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:

- 99.95% uptime
- Redundancy across Availability Zones

☒ Single-AZ DB instance deployment (1 instance)
 Creates a single DB instance without standby instances. This setup provides:

- 99.9% uptime
- No data redundancy

Write/read endpoint
 AZ 1
 Primary instance + SSD

Reader endpoints
 AZ 2
 Readable standby + SSD
 AZ 3
 Readable standby + SSD

Write/read endpoint
 AZ 1
 Primary instance

Standby (no endpoint)
 AZ 2
 Standby

Write/read endpoint
 AZ 1
 Primary instance

- Set the database identifier.
- Configure the master username and password.
- Attach the required security group.
- Create the database and wait for it to become available.

Step 2 – Connect directly from endpoint

- Open the **Amazon EC2** client instance.
- Connect to the instance using SSH.
- Install the PostgreSQL client if needed.

```
psql --version
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Shashank> psql --version
psql (PostgreSQL) 18.2
PS C:\Users\Shashank> curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
PS C:\Users\Shashank> psql "host=database-2.cghaxz7pgg8a.us-east-1.rds.amazonaws.com port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem"
Password for user postgres:

psql (18.2)
```

- Connect to the blue database endpoint.

```
psql -h <BLUE-ENDPOINT> -U <USERNAME> -d postgres
```

Step 3 – Create and Verify Sample Tables

- Create the sample database.

```
CREATE DATABASE seedingdemo;
```

- Connect to the sample database.

```
\c seedingdemo
```

- Create the table.

```
CREATE TABLE customer_info (
    customer_id BIGINT PRIMARY KEY,
    job_title VARCHAR(500),
    email_address VARCHAR(100),
    full_name VARCHAR(200),
    phone_number VARCHAR(20),
    city VARCHAR(50),
    state VARCHAR(50)
);
```

- Choose any sample csv and copy it.

```
\copy customer_info FROM 'C:/Users/Shashank/Downloads/customer_info.csv' WITH
(FORMAT csv, HEADER true, DELIMITER ',');
```

- Verify the records.

```
SELECT * FROM customer_info;
```

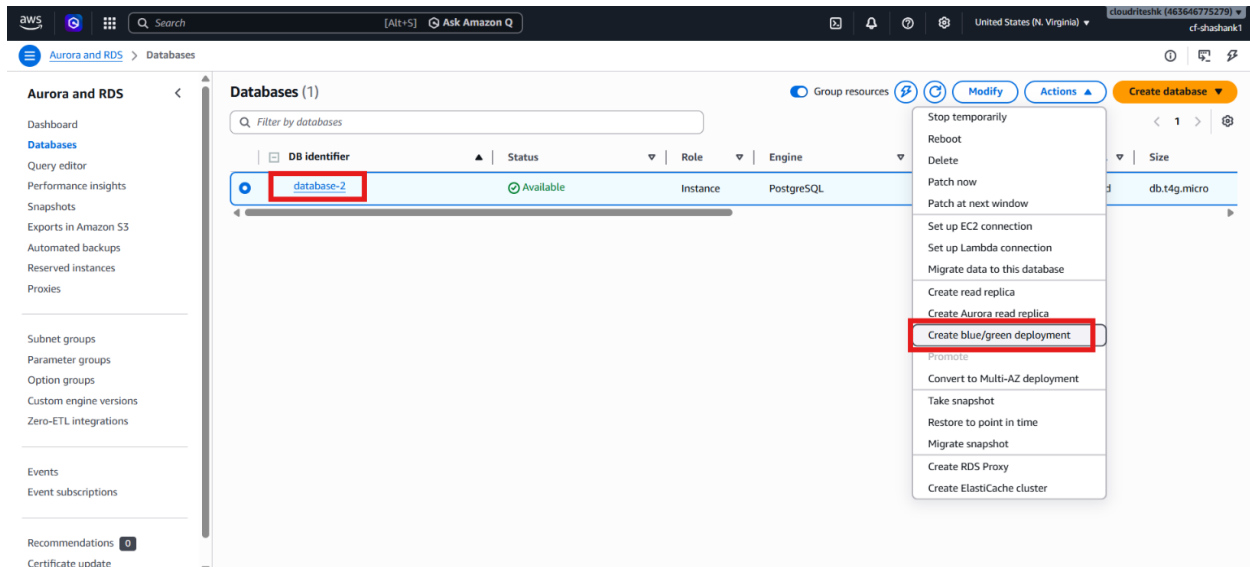
```
seedingdemo=> \copy customer_info FROM 'C:/Users/Shashank/Downloads/customer_info.csv' WITH (FORMAT csv, HEADER true);
COPY 5
seedingdemo=> SELECT * FROM customer_info;
```

customer_id	job_title	email_address	full_name	phone_number	city	state
1	Software Engineer	john.doe@example.com	John Doe	9876543210	Meerut	Uttar Pradesh
2	Data Analyst	jane.smith@example.com	Jane Smith	9123456780	Delhi	Delhi
3	Cloud Developer	rahul.verma@example.com	Rahul Verma	9988776655	Noida	Uttar Pradesh
4	Database Administrator	anita.sharma@example.com	Anita Sharma	9090909090	Jaipur	Rajasthan
5	Backend Developer	amit.kumar@example.com	Amit Kumar	9012345678	Lucknow	Uttar Pradesh

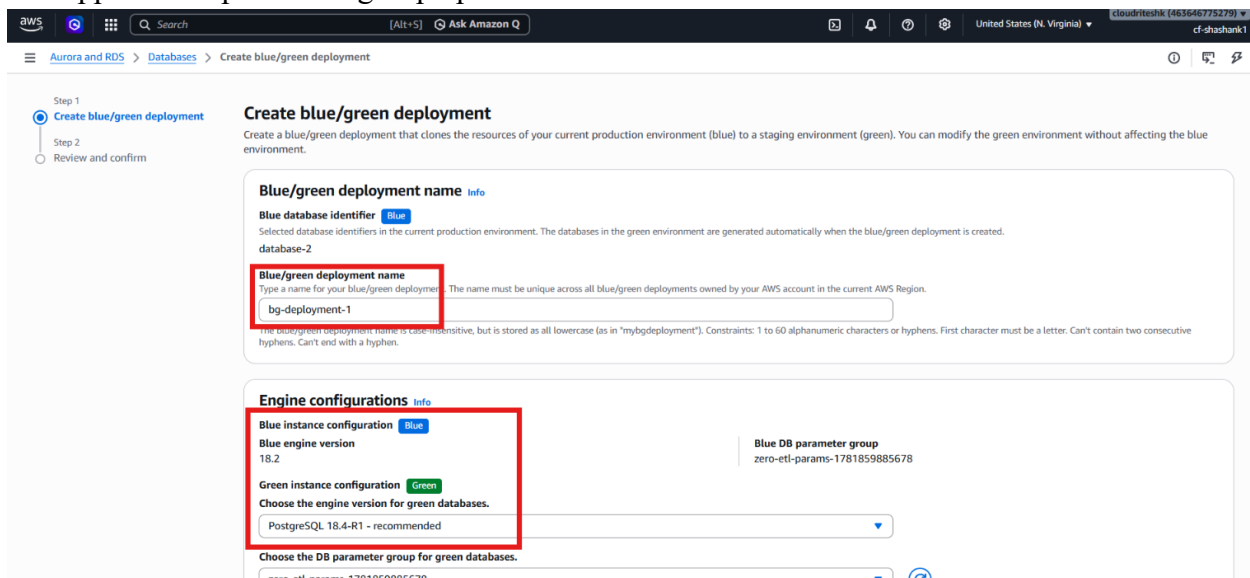
(5 rows)

Step 4 – Create the Blue/Green Deployment

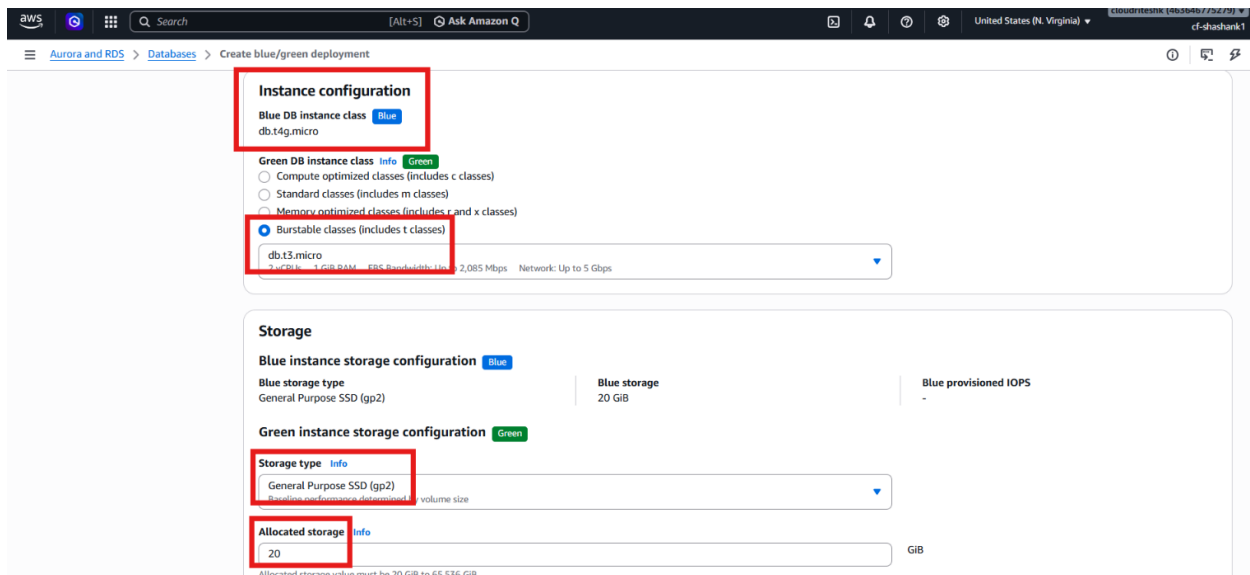
- Return to **Amazon RDS**.
- Select the blue database.
- Click **Actions**.
- Choose **Create Blue/Green Deployment**.



- AWS automatically selects the next major version for the green environment.
- Review the parameter group update notice.
- Approve the parameter group update.



- Choose the instance configuration as per your needs.
- For this lab I have chosen the lowest settings.



- Review settings once and click **Create**.

Step 5 – Verify the Green Environment

- Open the deployment details.
- Copy the green endpoint.
- Connect to the green database from the EC2 client.

```
psql -h <GREEN-ENDPOINT> -U <USERNAME> -d postgres
```

- Verify the database version on the green endpoint.

```
SELECT version();
```

- Confirm that the data was replicated to the green environment.
- Verify that the customer_info table still exists.

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Shashank>psql "host=database-2-green-jghmzj.cghaxz7pqg8a.us-east-1.rds.amazonaws.com port=5432 dbname=postgres user=postgres sslmode=verify-full ss
lrootcert=../global-bundle.pem"
Password for user postgres:

psql (18.2, server 18.4)
WARNING: Console code page (437) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> \c seedingdemo;
psql (18.2, server 18.4)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "seedingdemo" as user "postgres".
seedingdemo=> SELECT version();
          version
-----
PostgreSQL 18.4 on x86_64-pc-linux-gnu, compiled by x86_64-pc-linux-gnu-gcc (GCC) 12.4.0, 64-bit
(1 row)

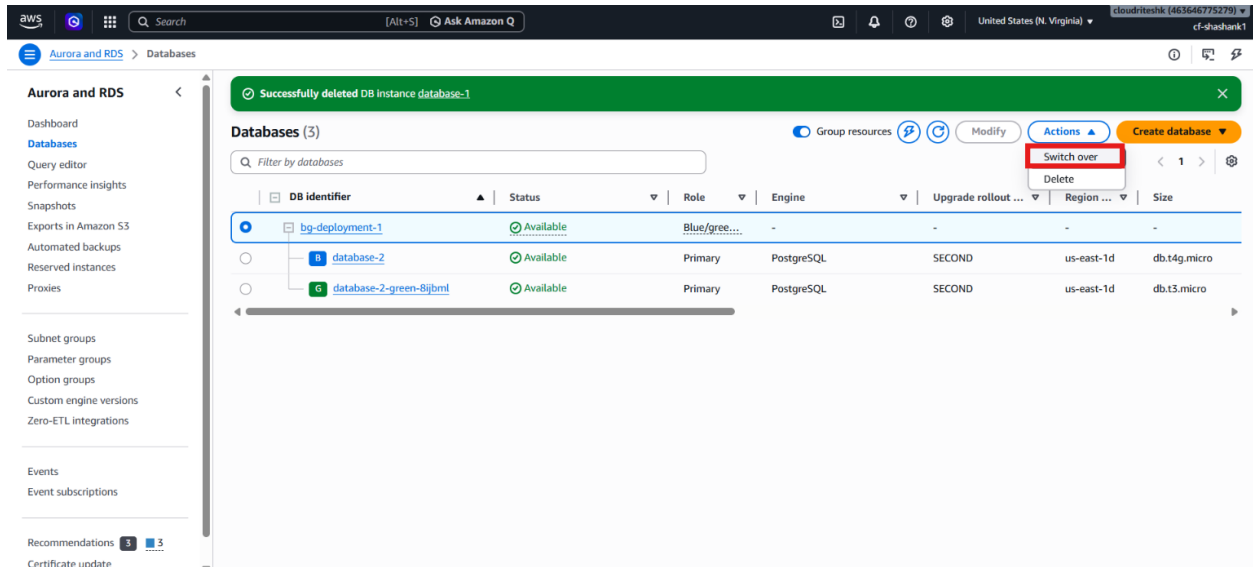
seedingdemo=> SELECT * FROM customer_info;
 customer_id | job_title | email_address | full_name | phone_number | city | state |  |
-----
1 | Software Engineer | john.doe@example.com | John Doe | 9876543210 | Meerut | Uttar Pradesh | 2 | Data Analyst
3 | Cloud Developer | rahul.verma@example.com | Rahul Verma | 9988776655 | Noida | Uttar Pradesh | 4 | Database Administrator
5 | Backend Developer | amit.kumar@example.com | Amit Kumar | 9012345678 | Lucknow | Uttar Pradesh | 5 rows
(5 rows)

seedingdemo=>
```

- Confirm that table contain the expected records.

Step 6 – Perform the Switchover

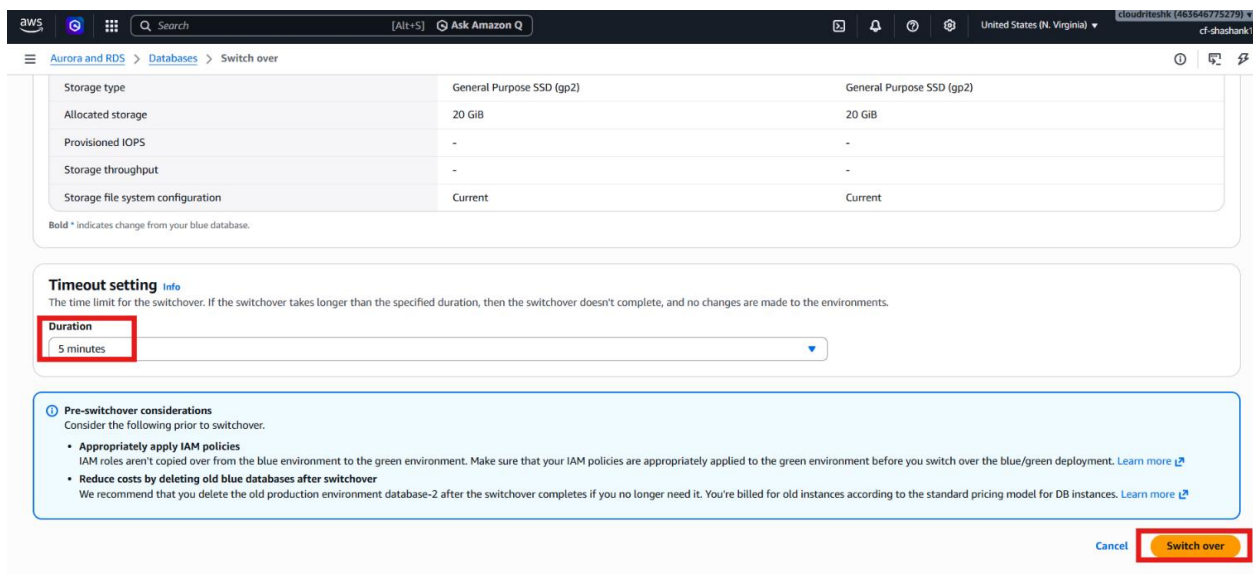
- Return to **Amazon RDS**.
- Open the blue/green deployment details.
- Click **Switch over**.



The screenshot shows the AWS Management Console interface for Amazon RDS. A green notification banner at the top states "Successfully deleted DB Instance database-1". The main content area displays a table of databases. The first row, "bg-deployment-1", is selected. The "Actions" menu is open, and the "Switch over" option is highlighted with a red box. The table columns include DB Identifier, Status, Role, Engine, Upgrade rollout, Region, and Size.

DB Identifier	Status	Role	Engine	Upgrade rollout	Region	Size
bg-deployment-1	Available	Blue/gree...	-	-	-	-
database-2	Available	Primary	PostgreSQL	SECOND	us-east-1d	db.t4g.micro
database-2-green-8ijbml	Available	Primary	PostgreSQL	SECOND	us-east-1d	db.t3.micro

- Set the timeout window.
- Confirm the switchover.



The screenshot shows the "Switch over" configuration page in the AWS Management Console. The page displays a comparison table of storage configurations between the current and target environments. Below the table, the "Timeout setting" section is visible, with the "Duration" dropdown menu set to "5 minutes" and highlighted with a red box. The "Pre-switchover considerations" section provides additional guidance, including applying IAM policies and reducing costs by deleting old blue databases after the switchover. The "Switch over" button is highlighted with a red box at the bottom right.

Storage type	General Purpose SSD (gp2)	General Purpose SSD (gp2)
Allocated storage	20 GiB	20 GiB
Provisioned IOPS	-	-
Storage throughput	-	-
Storage file system configuration	Current	Current

Timeout setting info
The time limit for the switchover. If the switchover takes longer than the specified duration, then the switchover doesn't complete, and no changes are made to the environments.

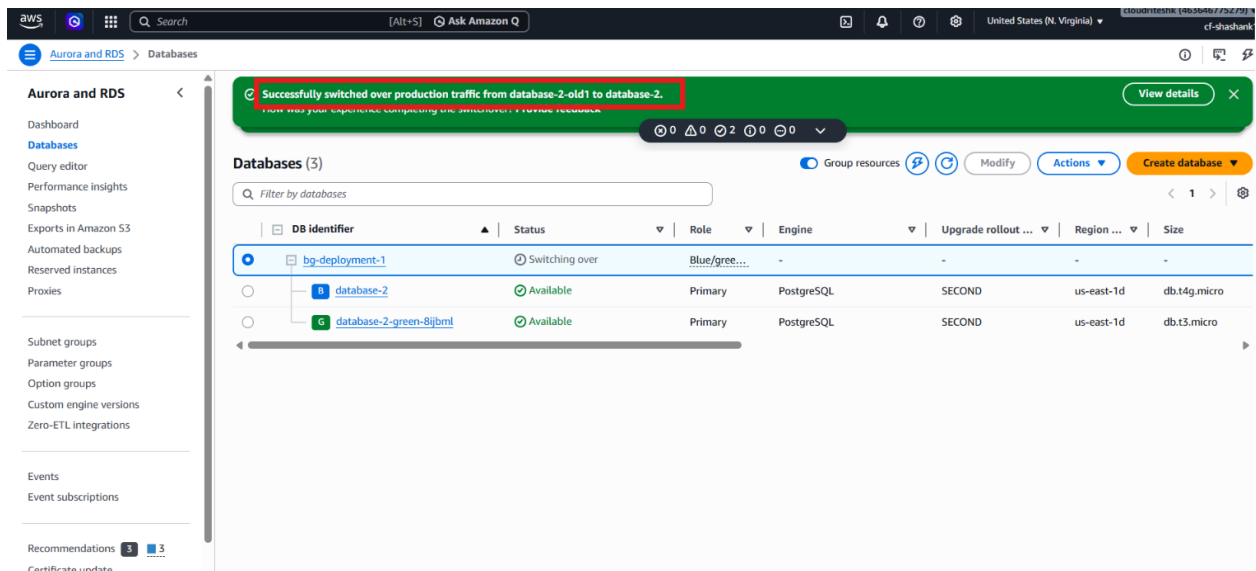
Duration
5 minutes

Pre-switchover considerations
Consider the following prior to switchover.

- **Appropriately apply IAM policies**
IAM roles aren't copied over from the blue environment to the green environment. Make sure that your IAM policies are appropriately applied to the green environment before you switch over the blue/green deployment. [Learn more](#)
- **Reduce costs by deleting old blue databases after switchover**
We recommend that you delete the old production environment database-2 after the switchover completes if you no longer need it. You're billed for old instances according to the standard pricing model for DB instances. [Learn more](#)

Cancel **Switch over**

- Wait for AWS to complete the switch.
- If the time provided is less for switchover then AWS will not switch to it and rollback.

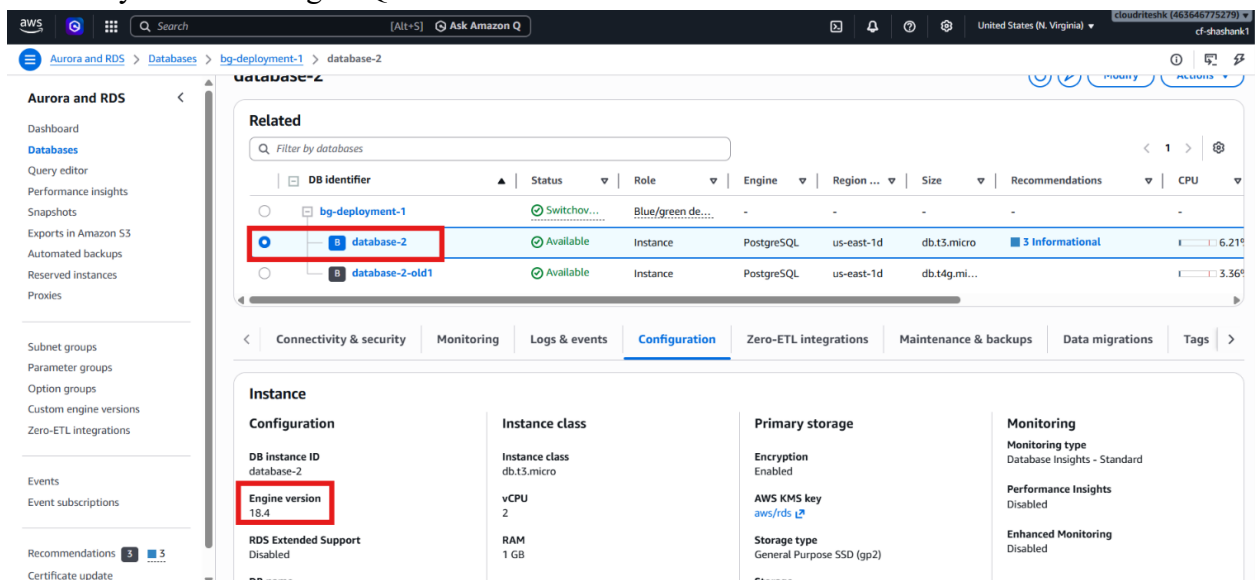


Step 7 – Verify the Upgraded Production Database

- Reconnect using the same client workflow.
- Use the production endpoint after the switchover.

```
psql -h <PRODUCTION-ENDPOINT> -U <USERNAME> -d postgres
```

- Verify the new PostgreSQL version.



```
SELECT version();
```

- Verify that the sample database still exists.

```
\c seedingdemo
```

- Confirm that table is present.

```
SELECT * FROM customer_info;
```

```
Command Prompt - psql "ho x + v
C:\Users\Shashank>psql "host=database-2.cghaxz7pqg8a.us-east-1.rds.amazonaws.com port=5432 dbname=postgres user=postgres
psql (18.2, server 18.4)
WARNING: Console code page (437) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> \c seedingdemo;
psql (18.2, server 18.4)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "seedingdemo" as user "postgres".
seedingdemo=> SELECT * FROM customer_info;
 customer_id | job_title | email_address | full_name | phone_number | city | state
-----+-----+-----+-----+-----+-----+-----
1 | Software Engineer | john.doe@example.com | John Doe | 9876543210 | Meerut | Uttar Pradesh
2 | Data Analyst | jane.smith@example.com | Jane Smith | 9123456780 | Delhi | Delhi
3 | Cloud Developer | rahul.verma@example.com | Rahul Verma | 9988776655 | Noida | Uttar Pradesh
4 | Database Administrator | anita.sharma@example.com | Anita Sharma | 9090909090 | Jaipur | Rajasthan
5 | Backend Developer | amit.kumar@example.com | Amit Kumar | 9012345678 | Lucknow | Uttar Pradesh
(5 rows)

seedingdemo=>
```

- Confirm that the upgraded green environment is now serving production traffic.

Verification

- The blue PostgreSQL environment was created successfully.
- The green environment was generated with the upgraded version.
- The source database was synchronized to the green environment.
- The customer_info table was preserved after the deployment.
- The customer_info table was also preserved after the deployment.
- The switchover completed successfully.
- The upgraded database became the active production environment.
- The downtime remained short and controlled during the upgrade.